

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: Seguridad Web y Autenticación con JWT
	Semana 12	Tutor: Ing. Victoria Castro

Introducción a la Seguridad Web:

¿Qué es la seguridad web?

La seguridad web es el conjunto de prácticas, tecnologías y políticas diseñadas para proteger los sitios web, aplicaciones, servidores y datos contra accesos no autorizados, modificaciones, interrupciones o destrucción. Su objetivo principal es garantizar la confidencialidad, integridad y disponibilidad (Tríada CIA) de la información.

Principales amenazas

- **XSS (Cross-Site Scripting):** Inyección de scripts maliciosos (generalmente JavaScript) en sitios web de confianza. El navegador del usuario ejecuta el script, lo que puede permitir al atacante robar cookies de sesión o tokens.
- **CSRF (Cross-Site Request Forgery):** Ataque que fuerza a un usuario final a ejecutar acciones no deseadas en una aplicación web en la que está autenticado actualmente, aprovechando la confianza que el sitio tiene en el navegador del usuario.
- **SQL Injection (SQLi):** Inyección de código SQL malicioso en los campos de entrada de una aplicación para manipular la base de datos subyacente, permitiendo leer, modificar o eliminar datos sensibles.
- **Robo de sesiones:** Intercepción o secuestro del identificador de sesión de un usuario (Session ID) para suplantar su identidad y acceder al sistema sin credenciales.

Importancia de autenticación y autorización

Son los pilares del control de acceso:

- **Autenticación:** Garantiza que el usuario es quien dice ser.
- **Autorización:** Determina a qué recursos o acciones tiene acceso ese usuario ya identificado. Sin un control estricto, cualquier usuario podría acceder a datos de administración o de otros clientes.

Sección 1: Conceptos Básicos de Autenticación

Autenticación vs Autorización

- **Autenticación (AuthN):** Responder a la pregunta *¿Quién eres?* (Ej. Ingresar usuario y contraseña).
- **Autorización (AuthZ):** Responder a la pregunta *¿Qué tienes permitido hacer?* (Ej. ¿Tiene este usuario permisos de administrador para borrar un post?).

Sesiones tradicionales (Stateful)

En el modelo tradicional, el servidor web crea una sesión para el usuario tras un login exitoso y guarda esa información en memoria o en una base de datos. Al usuario se le envía un Session ID almacenado en una cookie. En cada petición, el navegador envía la cookie y el servidor debe buscarla en su memoria para validar al usuario.

- *Desventaja:* Difícil de escalar horizontalmente (múltiples servidores necesitan compartir el estado de las sesiones).

Tokens (Stateless)

En la autenticación basada en tokens, el servidor no guarda el estado de la sesión. Tras el login, el servidor genera un token firmado criptográficamente y se lo envía al usuario. El cliente almacena el token y lo envía en cada petición. El servidor solo necesita validar la firma del token para saber si es legítimo, eliminando la necesidad de consultar una base de datos de sesiones.

Flujo de login moderno

1. El usuario envía sus credenciales (usuario/contraseña) al backend.
2. El backend verifica las credenciales contra la base de datos.
3. Si son correctas, el backend genera un Token (ej. JWT) firmado.
4. El backend responde al frontend enviando el token.
5. El frontend almacena el token de forma segura.
6. Para futuras peticiones a rutas protegidas, el frontend adjunta el token en las cabeceras (Headers).

Sección 2: Introducción a JWT

¿Qué es un JSON Web Token?

Un **JSON Web Token (JWT)** es un estándar abierto (RFC 7519) que define una forma compacta y autónoma para transmitir información de forma segura entre las partes como un objeto JSON. Al ser firmado, se puede verificar y confiar en la información.

Estructura del JWT

Un JWT se compone de tres partes separadas por puntos (.):

A. Header (Cabecera)

Contiene típicamente dos partes: el tipo de token (JWT) y el algoritmo de firma utilizado (como HMAC SHA256 o RSA).

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

B. Payload (Cuerpo/Carga útil)

Contiene los *claims* (afirmaciones), que son declaraciones sobre una entidad (normalmente, el usuario) y datos adicionales. Hay tres tipos de claims: registrados (estándar como sub, exp, iat), públicos y privados.

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

C. Signature (Firma)

Se toma el header codificado en Base64Url, el payload codificado en Base64Url, un secreto (secret key) proporcionado por el servidor, y se pasa por el algoritmo especificado en el header.

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  your-256-bit-secret  
)
```

Visualización de estructura

Visualmente, un JWT tiene la siguiente forma:

xxxxx.yyyyy.zzzzz

Donde:

- xxxxx es el Header codificado en Base64.
- yyyyy es el Payload codificado en Base64.
- zzzzz es la Firma (Signature).

Sección 3: Funcionamiento de JWT

[Cliente] --- (1) Login Credenciales ---> [Servidor]

[Cliente] <--- (2) Genera y Envía JWT ---- [Servidor]

[Cliente] --- (3) Petición + JWT Header -> [Servidor] (Valida firma sin consultar DB)

Generación del token: El backend valida las credenciales, junta los datos del usuario en el payload, define un tiempo de expiración y firma el conjunto con su clave privada/secreta.

Validación: Cuando llega una petición, el servidor toma el Header y el Payload recibidos, aplica la clave secreta y comprueba si el resultado coincide con la Firma enviada por el cliente. Si coincide y no ha expirado, la petición se procesa.

Expiración: Los JWT deben tener un tiempo de vida corto (ej. 15 minutos) definido en el claim exp. Una vez vencido, el token es rechazado automáticamente.

Refresh Tokens: Son tokens de larga duración (ej. 7 días) guardados de forma muy segura. Cuando el JWT principal (Access Token) expira, el frontend envía el *Refresh Token* a un endpoint especial para obtener un nuevo Access Token sin obligar al usuario a loguearse de nuevo.

Sección 4: Implementación Básica de JWT

Login de usuario: El cliente envía POST a /api/login con { email, password }.

Generación de token en backend: El backend usa una librería (ej. jsonwebtoken en Node.js, PyJWT en Python) para firmar el token utilizando una variable de entorno JWT_SECRET.

Envío al frontend: El servidor responde con un estado 200 OK devolviendo el token en el cuerpo JSON o, idealmente, configurándolo en una cookie.

Guardado del token: El cliente recibe la respuesta y almacena el token en la memoria de la aplicación o en el almacenamiento del navegador según la estrategia elegida.

Sección 5: Uso de JWT en Frontend

Guardar token: Pros y Contras

- **localStorage / sessionStorage:** Fácil de implementar mediante JavaScript (localStorage.setItem('token', jwt)).
 - **Riesgo:** Vulnerable a ataques XSS. Cualquier script malicioso inyectado puede leer el almacenamiento.

- **Cookies (HttpOnly):** Almacenamiento gestionado por el navegador. Si se activa la bandera HttpOnly, JavaScript **no** puede acceder a la cookie, protegiéndola de XSS.
 - *Riesgo:* Vulnerable a **CSRF** si no se configuran correctamente las propiedades SameSite y Secure.

Enviar Authorization Header

Para consumir una API protegida, el frontend debe adjuntar el token utilizando el esquema de autenticación HTTP Bearer:

Authorization: Bearer xxxxx.yyyyy.zzzzz

Sección 6: Protección de Rutas:

Rutas privadas: Vistas en el frontend (ej. un panel de usuario) que no deben renderizarse si no existe un token válido en el estado de la aplicación.

Middleware de autenticación: Funciones en el backend que interceptan las peticiones a endpoints protegidos, extraen el token del header Authorization, comprueban su validez y, si es correcto, permiten el paso a la función controladora.

Verificación de permisos / Roles de usuario: El payload del JWT puede contener el rol del usuario (ej. "role": "admin"). El middleware del backend lee este campo tras validar la firma y bloquea el acceso si el rol no coincide con los privilegios requeridos.

Sección 7: Seguridad en Almacenamiento de Tokens

Riesgos de localStorage: Al ser accesible por el objeto window, cualquier vulnerabilidad de scripts de terceros (CDNs, librerías comprometidas) puede resultar en la exfiltración masiva de tokens.

Uso de HttpOnly Cookies: Es el estándar recomendado para aplicaciones web comerciales. Almacenar el Access Token en memoria de la app y el Refresh Token en una cookie con las banderas HttpOnly, Secure (solo HTTPS) y SameSite=Strict o Lax.

Prevención de XSS: Sanitizar todas las entradas de usuario, implementar Content Security Policy (CSP) y evitar funciones peligrosas como eval() o innerHTML sin escapar.

Expiración de sesiones: Diseñar mecanismos para revocar tokens invalidando los Refresh Tokens en la base de datos cuando el usuario haga *Logout*.

Sección 8: Buenas Prácticas con JWT

Tokens cortos: Los Access Tokens deben expirar rápido (5 a 15 minutos) para mitigar el impacto si son robados.

HTTPS obligatorio: Toda la comunicación debe ir cifrada mediante TLS/HTTPS; de lo contrario, el token puede ser interceptado fácilmente en tránsito (Man-in-the-Middle).

Rotación de tokens: Implementar *Refresh Token Rotation*. Cada vez que se usa un Refresh Token, el servidor emite uno nuevo e invalida el anterior. Si un atacante roba un Refresh Token antiguo, el sistema detecta el doble uso y revoca el acceso por completo.

Validación del lado servidor: Nunca confíes en el frontend. Aunque el frontend bloquee vistas, el backend siempre debe validar el JWT en cada llamada API.

Nunca guardar información sensible en el payload: El payload está codificado en Base64, **no cifrado**. Cualquiera que intercepte el token puede decodificarlo con herramientas web y leer su contenido (nunca pongas contraseñas, datos bancarios o números de identificación ahí).

Sección 9: Ataques Comunes y Prevención

Ataque	Descripción	Prevención con JWT
Token Theft	Robo directo del token del almacenamiento del cliente.	Guardar tokens en Cookies HttpOnly y usar tiempos de expiración muy cortos.
Replay Attacks	El atacante intercepta un token válido y lo vuelve a enviar repetidamente para simular ser el usuario.	Usar HTTPS, tiempos de expiración cortos (exp) y claims únicos de un solo uso si es necesario (jti).
XSS	Inyección de código para capturar datos sensibles.	Sanitización con librerías (ej. DOMPurify), uso de cabeceras CSP fuertes y evitar localStorage para tokens de larga duración.
CSRF	El navegador envía automáticamente las cookies asociadas al dominio en una petición maliciosa.	Configurar cookies con la propiedad SameSite=Lax o Strict y usar cabeceras personalizadas que los scripts maliciosos entre sitios no puedan replicar fácilmente.
Fuga de credenciales	Exposición de llaves secretas de firma del servidor o payloads expuestos.	Proteger estrictamente el JWT_SECRET en variables de entorno seguras, rotar las llaves de firma periódicamente y no incluir datos confidenciales en el payload.

Bibliografía y Fuentes Oficiales

Para profundizar y contrastar la información técnica, consulta los siguientes recursos oficiales:

- OAuth 2.0 & JWT Security Best Practices: [IETF RFC 8725](#)
 - Documento oficial de ingeniería de la IETF detallando las vulnerabilidades y las defensas actuales en torno a JSON Web Tokens.
- JSON Web Token (JWT) Specification: [IETF RFC 7519](#)
 - El estándar original donde se definen los campos, la sintaxis y las reglas del ecosistema JWT.
- OWASP Top 10 (Open Worldwide Application Security Project): [OWASP Foundation Official Site](#)
 - Referencia global de seguridad que explica las vulnerabilidades más críticas de la web moderna como XSS, Inyecciones y Fallos de Autenticación.
- OWASP JSON Web Token Cheat Sheet: [OWASP Cheat Sheet Series](#)
 - Guía práctica oficial orientada a desarrolladores para la correcta implementación y configuración de seguridad en JWT.